

Introduction to instrumentation and field work

Introduction to instrumentation and field work

Field work and lab work - an introduction

Field measurements have historically been a difficult endeavour in fluid dynamics in general and oceanography in particular. Real world conditions makes it often very difficult to control the conditions of an experiment, and the large variations in weather and climate makes many phenomena local to possibly areas that are quite difficult to access. In particular, oceanographic measurements depend on systems that can be hundreds of kilometres wide, while simultaneously having processes that occur down at the millimetre scale (Kolmogorov dissipation etc..).

Historically, the only way to perform oceanographic measurements was to do them manually from a boat, and hope that the measurements taken are independent enough to form a picture of the oceanic state of interest. These methods date back centuries, with a noteworthy early example being the 1679-1680 expedition series performed by Count Luigi Fernando Marsili to measure sea water density in the eastern mediterranean ocean (<https://doi.org/10.1175/JPO-D-17-0168.1>).

The dynamic of oceanographic measurements got significantly easier following the advent of the computer and space travel. Satellite measurements allow for fully two-dimensional measurements of the ocean surface, while computers and portable electronics allow for the deployment of buoys and drifters that can sample without constant human input. The instruments were developed commercially and closed-source in the early stage of this latter kind of instrumentation, but these instruments were typically too expensive for mass deployment.

In this module, we will focus on the newest development in oceanographic measurements, open-source instrumentation. Cheap and programmable microcontrollers have made it possible to drastically cut costs, allowing for larger deployments and increased data collection. Furthermore, the move to open-source instrumentation facilitates customization to new scenarios.

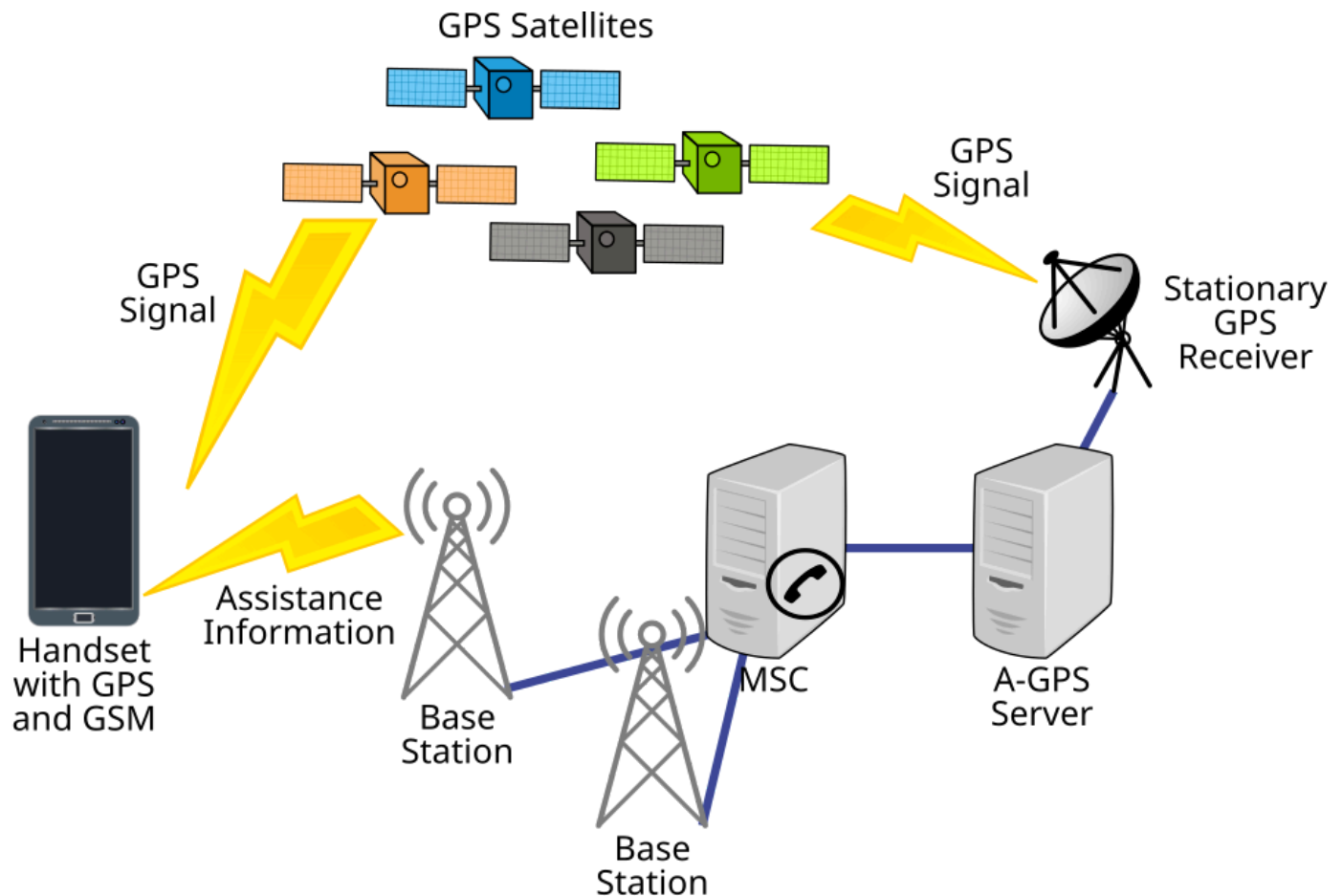
Some notable open source instruments include:

- [OpenCTD](#) - a cheap instrument for density measurements in the water column
- [microSWIFT](#) - Aircraft deployable wave sensor
- [OpenMetBuoy](#) wave buoy with satellite communication which can measure wave activity in hard-to-reach places.
- [Small Friendly Buoy](#) The focus of this lab module. High-frequency wave measuring device. Uses GSM (mobile communication) to transmit data
- [OLB](#) some shameless self-promotion, but these drifters measure position and temperature and uses radio communication to transmit buoy position in coastal environments.

The SFY

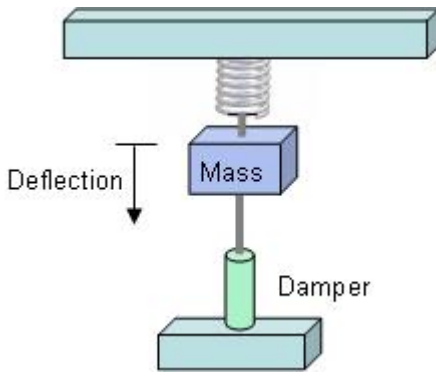
We will, in this module, use the Small Friendly Buoy (SFY) as our device to measure the oceanographic currents and wave activity in Drammen, Norway. The SFY is equipped with sensors that record both the buoy position (GPS) and the motion of the buoy in its own reference frame (IMU), which can then be used to compute statistics about the wave activity or the power spectrum in its entirety. A paper on the SFY can be read on the following DOI: <https://doi.org/10.1175/JTECH-D-23-0170.1>

GPS measurements - A short primer



The Global Positioning System GPS is an advanced geolocalization system that utilizes satellite telemetry and triangulation to determine the position of an object. The data from a GPS sensor is most often transmitted and stored in the form of [NMEA strings](#). While in everyday use, the most important information is the latitude/longitude of the object of interest, it is also able to estimate velocity and direction via the doppler shift of the messages between the GPS module and the satellite. Furthermore, the GPS satellites come equipped with an atomic clock, allowing for the use of a drift-free and accurate timestamp for measurements. The chief liabilities of a GPS is the limited accuracy (~3 m is common) and the high power demand.

IMUs - A somewhat longer short primer



The chief component so to say of the SFY is the *IMU - Inertial Measurement Unit*, and the onboard IMU is the [ISM330DHCX](#). These kinds of sensors come in two variants, namely either as a 6 *Degrees of Freedom (DoF)* or 9 *DoF* measurement device. A 6 DoF IMU comes with an onboard 3 dimensional accelerometer, a box filled with springs that measure the forces acting upon the IMU, and a gyroscope, a disc which can rotate freely and is designed to measure the rotation of the IMU. A 9 DoF IMU is a 6 DoF IMU which is also equipped with a magnetometer to measure the direction of the earth's magnetic field.

The IMU is of particular importance because it can sample at a high frequency and thereby create a very detailed image of the motion of the sensor, and thus by extension the motion of the instrument the IMU is attached to. These measurements are, however in the local reference frame of the IMU, and not in the global reference frame. Hence, it is in most application to adjust the measurements to a global reference frame. To do so, a Kalman filter is applied. A Kalman filter is a statistical algorithm which estimates a system state x_k from a series of measurements y_k through a dynamical system which presumes a known relation between y_k and x_k . As the SFY performs Kalman filtering onboard, we will not go into additional detail here, but if you are interested there are some excellent resources on the topic [here](#) and [here](#). **One important takeaway, however is that a Kalman filter for a 6DoF IMU can only estimate the global up-down axis, while a 9DoF IMU can also estimate the north-south and east-west accelerations acting upon the buoy.**

When it comes to wave measurements, IMUs are one of the most convenient ways we have to measure them. In particular, constructing the surface elevation η is done through a double-integration of the vertical acceleration a_z , although due to low-frequency noise inherent to IMU signals, direct integration of the signal is ill-advised. Instead, we can utilize the following identity of the fourier transform $\hat{a}_z = \mathcal{F}[a_z](\omega)$, namely

$$\frac{\partial \hat{a}_z}{\partial \omega} = i\omega \hat{a}_z.$$

As integrals and derivatives are each others inverse, we can therefore construct the fourier transformed surface elevation by

$$\hat{\eta} = \frac{\hat{a}_z}{(i2\pi f)^2}.$$

From here, a simple inverse Fourier transform yields us the surface elevation $\eta(t)$. It might seem, at this point like an unnecessary complication, yet the Fourier transformation from a temporal space to a

frequency space allows us to perform a clever high pass filter to eliminate the low-frequency noise. [Kohout et al. \(2015\)](#) performs this filter by multiplying the Fourier transform with the following integration weight function

$$R(f) = \begin{cases} 0, & f < f_1, \\ \frac{1}{2} \left[1 - \cos \left(\pi \frac{f-f_1}{f_2-f_1} \right) \right] \left[\frac{1}{(i2\pi f)^2} \right], & f_1 \leq f \leq f_2, \\ \frac{1}{(i2\pi f)^2}, & f_2 < f. \end{cases}$$

The two threshold frequencies were set by Kohout et al. to be $f_1 = 0.02$ Hz and $f_2 = 0.03$ Hz. The middle term is a small smoothing function, and is designed to ensure that the signal still is continuous. Once we have the filtered surface elevation η , it is then possible to compute important wave parameters such as the significant wave height, which is $H_s = 4\sigma_\eta$ four times the standard deviation of the surface elevation, the peak period T_p which is the wave period in the composite wave signal with the highest energy.

How to access the SFY data

The SFY uploads all data to a server called notehub, and comes with some prepackaged tools. We will finish this lecture by showing how to access the data from the SFY. To set everything up, open up a terminal and navigate to your documents folder (or wherever you want to work from). Then, we clone the project using

```
$ git clone git@github.com:gauteh/sfy.git
```

or alternatively, download the zip file by going to <https://github.com/gauteh/sfy>, then pressing the green Code button and in that drop-down menu click *Download zip*.

Once downloaded (and unzipped if you downloaded the zip file), navigate into the *sfy-processing* folder, and install the included scripts by writing

```
$ cd sfy/sfy-processing/  
$ conda env create -f ../environment.yml  
$ conda activate sfy  
$ pip install -e .
```

in your terminal. Keep in mind that the installation procedures here are written with linux in mind. If you have a windows PC, you might need to activate [Windows Subsystem for Linux](#). Once the conda environment has finished installing, we can now proceed to set up the login-credentials necessary to access the data. On linux, this can be easily done by writing

```
echo "export SFY_SERVER='https://wavebug.met.no'" >> ~/.bashrc  
echo "export SFY_READ_TOKEN='secret'" >> ~/.bashrc # replace with the actual  
token, ask your lecturer  
echo "export SFY_DATA_CACHE='/tmp/sfy-cache'" >> ~/.bashrc
```

```
source ~/.bashrc
conda activate sfy
```

While on mac, the commands are

```
echo "export SFY_SERVER='https://wavebug.met.no'" >> ~/.zshrc
echo "export SFY_READ_TOKEN='secret'" >> ~/.zshrc # replace with the actual
token, ask your lecturer
echo "export SFY_DATA_CACHE='/tmp/sfy-cache'" >> ~/.zshrc
source ~/.zshrc
conda activate sfy
```

The secret SFY token will be given in class, or is available upon request to `larswd@math.uio.no`.

Now, we can download data from your buoy by writing the following command in your terminal window:

```
sfydata axl ts UIO_MEK_<your_buoy_name> --file <outfilename>.nc
```

This will fetch all data pertaining to your buoy of choice from the last 24 hours that are on the server and out to the file specified at the end of the command. If you need to look further back than 24 hours, you can amend the command with the `--tx-start` flag, whereupon you can specify the date from which you wish to download the files, for instance:

```
sfydata axl ts UIO_MEK_<your_buoy_name> --file <outfilename>.nc --tx-start
2026-05-04
```

will download all files related to the buoy from the 4th of may and onward. If you want to specify an end date, that can be achieved by the flag `--tx-end`.

This netCDF file can most easily be read and processed using [xarray](#) that come preinstalled in the sfy conda environment. We will not cover the functionality of xarray in detail, but this script will read the netcdf file, extract the vertical acceleration, plot it and convert it to a numpy array if that is more convenient.

```
import xarray as xr
import matplotlib.pyplot as plt
import sys

filename = sys.argv[1]

#read file
ds = xr.open_dataset(filename)

#print metadata
print(ds)

#extract vertical acceleration
```

```
w_z = ds.w_z
timestamps = w_z.time

#plot acceleration
w_z.plot.line()
plt.show()

#extract to numpy
np_w_z = w_z.to_numpy()
```

Do note that the wave spectrum can relatively easily be computed using [Welch's method](#) using the [csd function from SciPy](#).

Finally, all documentation on the sfydata fetching tool can be found by using the help command

```
$ sfydata axl ts UI0_MEK_<your_buoy_name> --help
```

Fetching GPS data

The built-in GPS does not work without a dedicated patch-antenna, which will be handed out in class later. To download your GPS trace, you can use the following command

```
$ sfydata track csv UI0_MEK_<your_buoy_name >> <my_trace_file>.csv
```

With the following python script (`make_map.py`), you can then plot the trajectories taken by the buoy:

```
import pandas as pd
import matplotlib.pyplot as plt
import plotly
import plotly.express as px
import plotly.graph_objects as go
import numpy as np
import sys

infile = sys.argv[1]

data = pd.read_csv(infile)

fig_buoys = px.scatter_map(data, lat="Latitude", lon="Longitude")
fig = go.Figure(data = fig_buoys)
fig.update_layout(mapbox_style="open-street-map")
fig.show()
```

by writing the following in the terminal

```
$ python make_map.py <my_trace_file>.csv
```

While you can use the following utm function to get *xy*-coordinates that you can work with on a cartesian grid:

```
import utm
import pandas as pd

data = pd.read_csv(infile)
lats = data["Latitude"]
lons = data["Longitude"]

u = utm.from_latlon(lats, lons)
x = u[0]
y = u[1]
```

Exercise statements

This module is meant to give an introduction to field measurements and buoy technology. The students are expected to:

- Validate each sensor component of the SFY.
 - What is the accuracy of the GPS, how confident can you be in the buoy position and movements. How can you validate the position?
 - How accurate is the SFY IMU in capturing motion? Can you reasonably reconstruct controlled movement? Can you capture wave motion accurately? How can you validate the IMU motion?
- Plan and prepare for a field deployment.
 - Check if the container is water proof.
 - Compute and add sufficient ballast if necessary
 - Plan for where to deploy the buoys
- Analyze field data
 - Identify the changes in significant waveheight during the field deployment
 - Did the drifters move as expected?